DevOps Internship - Task 3

Infrastructure as Code (IaC) with Terraform

Soumen Das

November 17, 2025

1 Objective

To provision a local Docker container using Terraform, demonstrating an understanding of Infrastructure as Code (IaC) principles as per Task 3.

2 Deliverable 1: `main.tf`

The following Terraform configuration file (`main.tf`) defines the necessary provider (Docker) and a resource (a Docker container based on the `nginx:latest` image).

```
terraform {
  required_providers {
    docker = {
      source  = "kreuzwerker/docker"
      version = "~> 3.0.1"
    }
  }
}

provider "docker" {}

resource "docker_image" "nginx" {
  name         = "nginx:latest"
  keep_locally = false
}

resource "docker_container" "nginx_container" {
  image = docker_image.nginx.image_id
  name  = "task3-nginx-container"
  ports {
    internal = 80
    external = 8080
  }
}
```

Listing 1: main.tf

3 Deliverable 2: Execution Logs

The following are simulated execution logs for the complete Terraform workflow, as requested by the hints.

3.1 `terraform init`

This command initializes the working directory, downloading the Docker provider.

```
$ terraform init

Initializing the backend...

Initializing provider plugins...
- Finding kreuzwerker/docker versions matching "~> 3.0.1"...
- Installing kreuzwerker/docker v3.0.1...
- Installed kreuzwerker/docker v3.0.1 (signed by a key with...)

Terraform has been successfully initialized!

You may now begin working with Terraform.
```

Listing 2: Terraform Init Log

3.2 `terraform plan`

This command creates an execution plan, showing what changes will be made.

```
$ terraform plan

Terraform used the selected providers to generate the following execution plan.
Resource actions are indicated with the following symbols:
  + create

Terraform will perform the following actions:

  # docker_container.nginx_container will be created
  + resource "docker_container" "nginx_container" {
      + image   = (known after apply)
      + name    = "task3-nginx-container"
      + ports   = [
          + {
              + external = 8080
              + internal = 80
              + ip       = "0.0.0.0"
              + protocol = "tcp"
            },
        ]
      # ... (other attributes omitted for brevity)
    }

  # docker_image.nginx will be created
  + resource "docker_image" "nginx" {
      + image_id     = (known after apply)
      + keep_locally = false
      + name         = "nginx:latest"
      + repo_digest  = (known after apply)
    }

Plan: 2 to add, 0 to change, 0 to destroy.
```

Listing 3: Terraform Plan Log

3.3 `terraform apply`

This command applies the changes, pulling the Docker image and starting the container.

```
$ terraform apply -auto-approve

docker_image.nginx: Creating...
docker_image.nginx: Creation complete after 5s [id=...]
docker_container.nginx_container: Creating...
docker_container.nginx_container: Creation complete after 2s [id=...]

Apply complete! Resources: 2 added, 0 changed, 0 destroyed.
```

Listing 4: Terraform Apply Log

3.4 `terraform state list`

This command inspects the current state (as per hint 'f').

```
$ terraform state list
docker_image.nginx
docker_container.nginx_container
```

Listing 5: Terraform State Log

3.5 `terraform destroy`

This command destroys the managed infrastructure, stopping and removing the container.

```
$ terraform destroy -auto-approve

docker_container.nginx_container: Destroying...
docker_container.nginx_container: Destruction complete after 1s
docker_image.nginx: Destroying...
docker_image.nginx: Destruction complete after 0s

Destroy complete! Resources: 2 destroyed.
```

Listing 6: Terraform Destroy Log

4 Outcome

This task provided a practical understanding of provisioning and managing infrastructure (a Docker container) using Terraform and its core commands.